Robot CLI Command Tool

1. Course Content

Course Overview

- This section covers the usage of the Robot CLI (Command Line Interface) command tool. Through the command line interface, users can directly control the robot's Ackermann chassis, 6-DOF robotic arm, vision algorithms, and navigation mapping in the terminal without launching a large graphical interface or relying on AI large models.
- The CLI command tool directly encapsulates all MCP Tool interfaces of the `ackermann_mcp` server. It is not only an efficient tool for development, debugging, and unit testing, but also serves as the standard underlying interface directly called by upper-layer applications like OpenClaw.

Learning Objectives

1. Understand the architectural positioning and application scenarios of the Robot CLI command tool
2. Master the parameter formats of common CLI commands and control methods for Ackermann chassis/robotic arm
3. Master the operation process for independently verifying the robot's core functions via command line

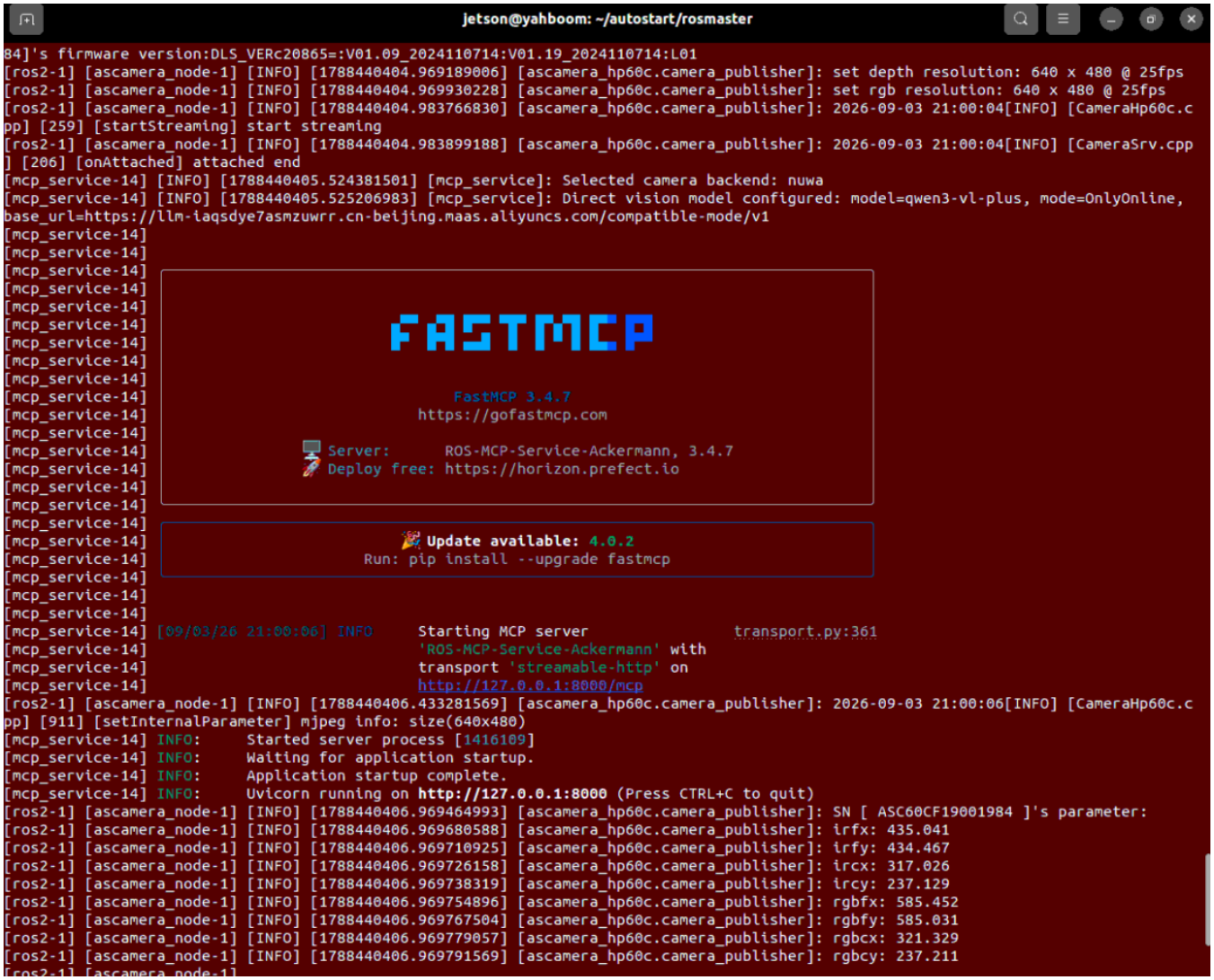
> **Important**
>
> This section is designed for the **A1Pro Ackermann Robot with 6-DOF Robotic Arm**, fully supporting chassis Ackermann kinematics control and robotic arm multi-joint/vision-based grasping control.

---

2. Prerequisites

Before running any CLI commands, ensure that the robot's backend MCP service is already started:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```



The service listens for client connections on `http://localhost:8000/mcp` by default.

---

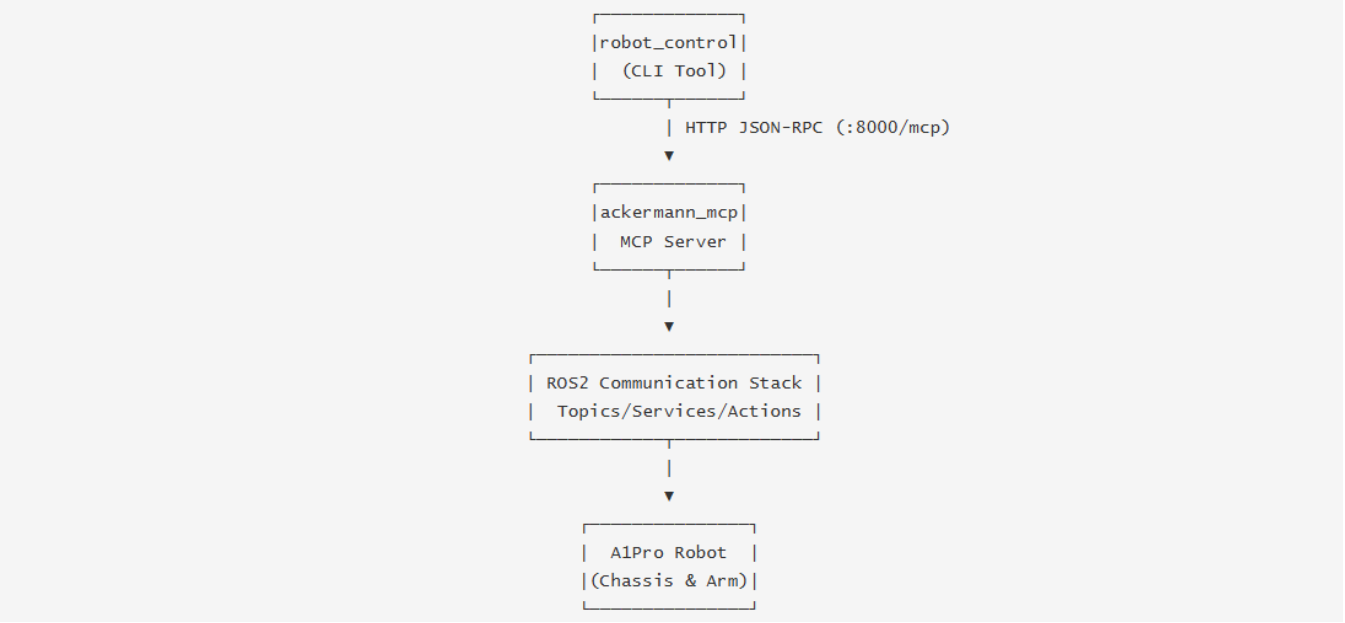
3. CLI Tool Introduction

The Robot CLI control tool is a client program developed based on the Python `cyclopts` modern command-line framework. The executable is located at:
`dofbot_pro_ws/src/ackermann_mcp/cli/robot_control`

The CLI tool internally connects to the local FastMCP server via `fastmcp.Client` asynchronously, transparently encapsulating user commands entered in the terminal into MCP JSON-RPC requests and echoing the execution results.



4. CLI Command Control Communication Architecture



5. CLI Command List and Usage Guide

5.1 Command Format Specification

CLI commands follow a standard hierarchical subcommand invocation syntax:

```
robot_control call-tool <command-name> --param1 <value1> --param2 <value2> ...
```

- `robot_control`: Main program name
- `call-tool`: Tool invocation subcommand group
- `<command-name>`: The specific tool name registered with the MCP server (note the capitalization, e.g.,

`Move`, `ControlSixArmJoint`, `SeeWhat`)

- `--param <value>`: The specific parameter key-value pair

> **Tip**
>
> You can always enter the following command in the terminal to list all tools registered with the current server and their parameter signatures:
>
> ```
> robot_control list-tools
> ```

```
jetson@yahboom:~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/cli$ robot_control list-tools
In USB-camera mode this unsupported step returns success without moving the robot.

  PlaceWaste()
    Place the waste object.
In USB-camera mode this unsupported step returns success without moving the robot.

  TTS(text: string)
    Robot text-to-speech

  Rotate(angle: number)
    Rotates in place by a specified angle.
For Ackermann robot, this uses low-speed forward movement + turning.

  TargetTrack(x1_or_cmd: string, y1: string = ..., x2: string = ..., y2: string = ...)
    Visual tracking of target objects

  GetTargetDist(query: string)
    Obtain the distance from the target to the Ackermann robot based on object descriptions.
In USB-camera mode this unsupported step returns success without estimating distance.

  GetAprilTagIDs()
    Acquire AprilTag IDs present in the current view. USB mode uses one fresh frame.

  GraspArUcoTarget(target_id: integer)
    Grasp objects marked with AprilTags. Ackermann chassis uses forward steering instead of lateral strafing.

  FollowLine(color: string)
    Follow a colored line on the ground. Supported colors: red, green, blue, yellow. Ackermann-safe low-speed
turning.

jetson@yahboom:~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/cli$
```

5.2 Core CLI Commands Summary Table

Category	Tool Name (Command)	Common Parameters	Function Description
Chassis Movement	`Move`	`--distance`, `--direction`	Move the specified distance in meters in the specified direction (`forward`, `backward`, `left`, `right`)
Chassis Movement	`MoveWithSpeed`	`--linear-x`, `--angular-z`, `--duration`	Travel for the specified duration at the given linear and angular velocities

Category	Tool Name (Command)	Common Parameters	Function Description
Chassis Movement	`AdjustChassisFitArmRange`	`--x`, `--y`, `--z`	Automatically fine-tune the chassis pose so that the spatial target coordinates fall within the optimal grasping reachable range of the robotic arm
Arm Control	`InitArmPose`	None	Reset the 6-DOF robotic arm to the factory default inspection pose
Arm Control	`ControlSixArmJoint`	`--arm-joint-1` ~ `--arm-joint-6`, `--runtime`	Simultaneously control joints 1-6 to specified angles (0-180 degrees)
Arm Control	`ControlSingleArmJoint`	`--arm-joint-id`, `--angle`, `--runtime`	Individually adjust the specified servo angle (1-6)
Arm Control	`GetCurrentArmJointAngles`	None	Read the actual angle values of all 6 joints
Arm Control	`GetEndEffectorPose`	None	Read the 6DoF spatial pose of the robotic arm end effector (X, Y, Z, Euler angles)
Smart Grasping	`GraspArUcoTarget`	`--target-id`	Perform Ackermann adaptive approach and closed-loop grasping for AprilTag blocks in view
Smart Grasping	`Pick`	`--x1`, `--y1`, `--x2`, `--y2`	Perform dive-and-grasp based on image target bounding box pixel coordinates
Smart Grasping	`Place`	`--place-x`, `--place-y`, `--place-z`	Precisely place the gripped object to the specified 3D position in the world coordinate system
Smart Grasping	`GraspWaste` / `PlaceWaste`	`--waste-name`	Grasp/place the specified type of waste object
Vision & Perception	`SeeWhat`	None	Capture a frame of the current camera view and save it for display
Vision & Perception	`AdjustCameraView`	`--pitch`, `--yaw`	Adjust the camera mounting pose (pitch and yaw)

Category	Tool Name (Command)	Common Parameters	Function Description
Vision & Perception	`GetBbox`	`--query`	Input natural language description to detect and return the target bounding box pixel coordinates
Vision & Perception	`GetPlacePoint`	`--query`	Input placement description to detect and return a suitable 3D spatial coordinate point for placement
Vision & Perception	`GetTargetDist`	`--query`	**Based on the natural language description of object features, calculate the actual 3D physical distance from the target to the robot vehicle (meters)**
Vision & Perception	`GetAprilTagIDs`	None	Detect and list all visible AprilTag IDs in the current field of view
Vision & Perception	`GetWasteRecognitionResults`	None	Get the waste classification results identified by the YOLO deep learning model in the current field of view
Vision & Perception	`TargetTrack`	`--x1-or-cmd`, `--y1`, `--x2`, `--y2`	Start/cancel (`cancel`) target visual servo tracking
Line Following Navigation	`FollowLine`	`--color`	Travel along the colored track on the ground (`red`, `green`, `blue`, `yellow`)
Line Following Navigation	`Navigation`	`--location`	Autonomously navigate to the target point symbol in the map mapping table
Line Following Navigation	`RecordMapLocation`	`--name`, `--symbol`	Record the robot's current parking pose and bind it to a map symbol
Line Following Navigation	`GetMapMapping`	None	Query all recorded locations and symbol mapping dictionary

Category	Tool Name (Command)	Common Parameters	Function Description
System Interaction	`TTS`	`--text`	Control the onboard speaker to perform text-to-speech synthesis and broadcast

6. Demonstration Examples

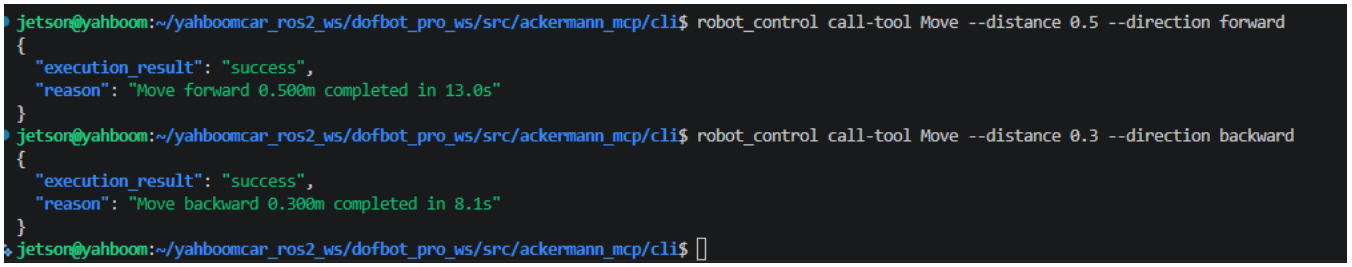
Example 1: Chassis Translation Movement

Make the robot move forward 0.5 meters:

```
robot_control call-tool Move --distance 0.5 --direction forward
```

Make the robot reverse 0.3 meters:

```
robot_control call-tool Move --distance 0.3 --direction backward
```



Example 2: Robotic Arm Pose Reset and Multi-Joint Control

Reset the robotic arm to the inspection pose with one key:

```
robot_control call-tool InitArmPose
```

Simultaneously control all 6 joints to working position (base 90 degrees, gripper closed):

```
robot_control call-tool ControlSixArmJoint --arm-joint-1 90 --arm-joint-2 130 --arm-joint-3
20 --arm-joint-4 20 --arm-joint-5 90 --arm-joint-6 0 --runtime 1500
```

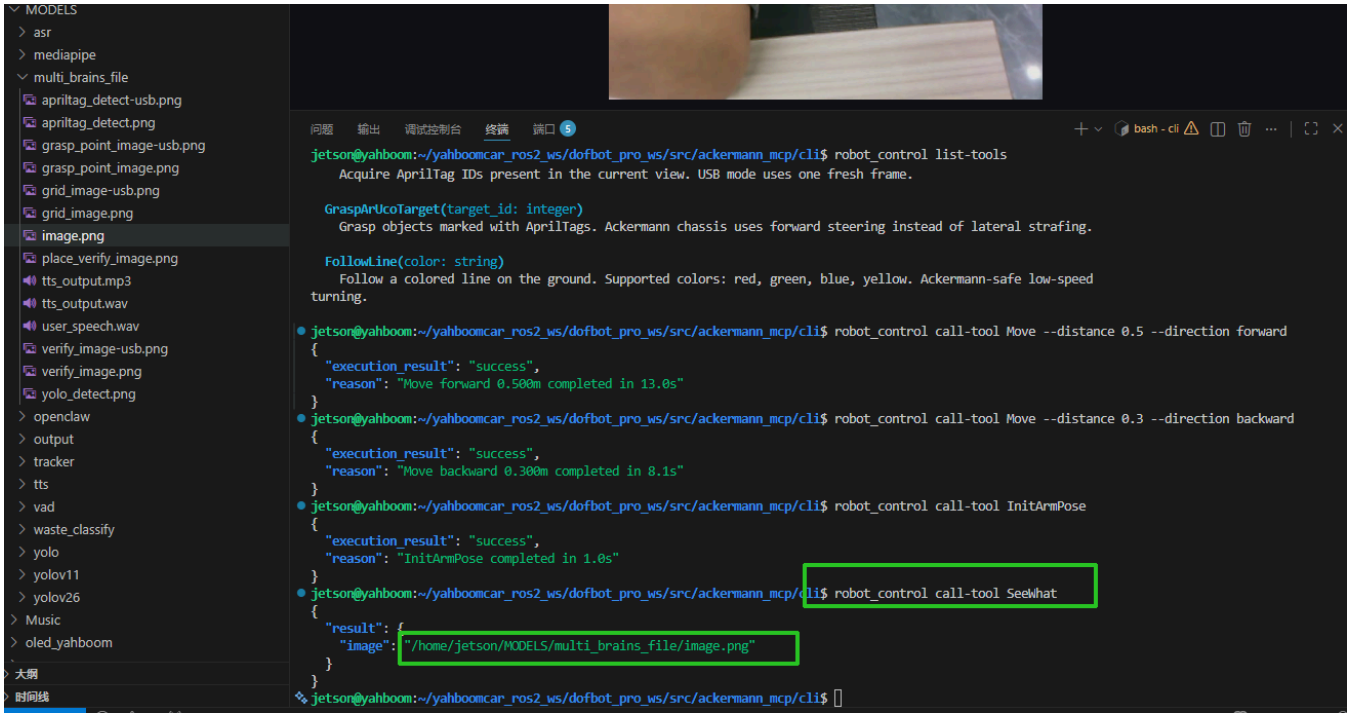
Individually open the gripper (joint 6 opens to 180 degrees):

```
robot_control call-tool ControlSingleArmJoint --arm-joint-id 6 --angle 180 --runtime 1000
```

Example 3: Environment Capture and AprilTag Label Scanning

Get the current camera first-person view and save it:

```
robot_control call-tool SeeWhat
```



Detect the AprilTag label numbers present in the current field of view:

```
robot_control call-tool GetAprilTagIDs
```

Automatically track and grasp the No. 2 AprilTag block on the desk:

```
robot_control call-tool GraspArUcoTarget --target-id 2
```

Example 4: Natural Language Target Ranging (`GetTargetDist`) and Visual Positioning

Input natural language description to calculate the real-time spatial physical distance from a specific object in front to the robot vehicle (meters):

```
robot_control call-tool GetTargetDist --query "red block"
```

Combine with the vision perception model to return the 2D pixel bounding box (bbox) of the object:

```
robot_control call-tool GetBbox --query "red block"
```

---

7. Source Code Analysis (`robot_control`)

Source Code Path: `dofbot_pro_ws/src/ackermann_mcp/cli/robot_control`

The CLI tool is built on `cyclopts` and implements interaction with the MCP server through a lightweight asynchronous client:

7.1 Core Service Communication Implementation

```
# robot_control core communication logic
import cyclopts
from fastmcp import Client
from rich.console import Console

# Specify MCP server listening address
CLIENT_SPEC = 'http://localhost:8000/mcp'

app = cyclopts.App(name="ackermann-robot", help="CLI for Ackermann MCP server")
call_tool_app = cyclopts.App(name="call-tool", help="Call a tool on the server")
app.command(call_tool_app)
console = Console()

async def _call_tool(tool_name: str, arguments: dict) -> None:
    # Filter out None-valued empty parameters
    filtered = {
        k: v for k, v in arguments.items()
        if v is not None and (not isinstance(v, list) or len(v) > 0)
    }
    # Establish session connection with FastMCP server and invoke specified tool
    async with Client(CLIENT_SPEC) as client:
        result = await client.call_tool(tool_name, filtered, raise_on_error=False)
        _print_tool_result(result)
```

7.2 Tool Command Decorator Wrapper

Each subcommand uses type annotations to strictly define the input format. For example, the robotic arm grasping AprilTag block and target ranging tool:

```
# GraspArUcoTarget command wrapper
@call_tool_app.command(name='GraspArUcoTarget')
async def GraspArUcoTarget(
    *,
    target_id: Annotated[int, cyclopts.Parameter(help="ID of the AprilTag to grasp")],
) -> None:
    '''Grasp objects marked with AprilTags using Ackermann-compatible approach motion'''
    await _call_tool('GraspArUcoTarget', {'target_id': target_id})

# GetTargetDist natural language spatial ranging command wrapper
@call_tool_app.command(name='GetTargetDist')
async def GetTargetDist(
    *,
```

```
    query: Annotated[str, cyclopts.Parameter(help="Description of the target object's
position and visual characteristics in the current view")],
) -> None:
    '''Obtain the distance from the target to the robot vehicle based on object
descriptions'''
    await _call_tool('GetTargetDist', {'query': query})
```

Through this declarative design, the CLI provides both a self-documenting `--help` system and ensures that the command format sent to the underlying ROS 2 is completely consistent with the format used by large model invocations. The `GetTargetDist` receives the user's semantic description and links with the server-side vision multimodal localization algorithm and depth camera point cloud to directly calculate the actual spatial straight-line physical distance between the specified natural object and the robot.